



Version 9.1-DE, Dezember 2025: (basiert auf der AsciiDoc Version)

Inhaltsverzeichnis

1. Einführung und Ziele	2
1.1. Aufgabenstellung	2
1.2. Qualitätsziele	2
1.3. Stakeholder	3
2. Randbedingungen	5
3. Kontextabgrenzung	6
3.1. Fachlicher Kontext	6
3.2. Technischer Kontext	7
4. Lösungsstrategie	8
5. Bausteinsicht	9
5.1. Whitebox Gesamtsystem	11
5.1.1. <Name Blackbox 1>	12
5.1.2. <Name Blackbox 2>	12
5.1.3. <Name Blackbox n>	12
5.1.4. <Name Schnittstelle 1>	12
5.1.5. <Name Schnittstelle m>	12
5.2. Ebene 2	13
5.2.1. Whitebox <Baustein 1>	13
5.2.2. Whitebox <Baustein 2>	13
5.2.3. Whitebox <Baustein m>	13
5.3. Ebene 3	13
5.3.1. Whitebox <_Baustein x.1_>	13
5.3.2. Whitebox <_Baustein x.2_>	13
5.3.3. Whitebox <_Baustein y.1_>	14
6. Laufzeitsicht	15
6.1. <Bezeichnung Laufzeitszenario 1>	15
6.2. <Bezeichnung Laufzeitszenario 2>	16
6.3. <Bezeichnung Laufzeitszenario n>	16
7. Verteilungssicht	17
7.1. Infrastruktur Ebene 1	17
7.2. Infrastruktur Ebene 2	18
7.2.1. <Infrastrukturelement 1>	18
7.2.2. <Infrastrukturelement 2>	18
7.2.3. <Infrastrukturelement n>	18
8. Querschnittliche Konzepte	19
8.1. <Konzept 1>	20
8.2. <Konzept 2>	20
8.3. <Konzept n>	20

9. Architekturentscheidungen	21
10. Qualitätsanforderungen	22
10.1. Übersicht der Qualitätsanforderungen	22
10.2. Qualitätsszenarien	23
11. Risiken und technische Schulden	25
12. Glossar	26

Über arc42

arc42, das Template zur Dokumentation von Software- und Systemarchitekturen.

Template Version 9.1-DE. (basiert auf der AsciiDoc Version), Dezember 2025

Created, maintained and © by Dr. Peter Hruschka, Dr. Gernot Starke and contributors. Siehe <https://arc42.org>.

HINWEIS

Diese Version des Templates enthält Hilfen und Erläuterungen. Sie dient der Einarbeitung in arc42 sowie dem Verständnis der Konzepte. Für die Dokumentation eigener System verwenden Sie besser die *plain* Version.

Chapter 1. Einführung und Ziele

Beschreibt die wesentlichen Anforderungen und treibenden Kräfte, die bei der Umsetzung der Softwarearchitektur und Entwicklung des Systems berücksichtigt werden müssen.

Dazu gehören:

- zugrunde liegende Geschäftsziele,
- wesentliche Aufgabenstellungen,
- wesentliche funktionale Anforderungen,
- Qualitätsziele für die Architektur und
- relevante Stakeholder und deren Erwartungshaltung.

1.1. Aufgabenstellung

Inhalt

Kurzbeschreibung der fachlichen Aufgabenstellung, treibenden Kräfte, Extrakt (oder Abstract) der Anforderungen. Verweis auf (hoffentlich vorliegende) Anforderungsdokumente (mit Versionsbezeichnungen und Ablageorten).

Motivation

Aus Sicht der späteren Nutzung ist die Unterstützung einer fachlichen Aufgabe oder Verbesserung der Qualität der eigentliche Beweggrund, ein neues System zu schaffen oder ein bestehendes zu modifizieren.

Form

Kurze textuelle Beschreibung, eventuell in tabellarischer Use-Case Form. Sofern vorhanden, sollte die Aufgabenstellung Verweise auf die entsprechenden Anforderungsdokumente enthalten.

Halten Sie diese Auszüge so knapp wie möglich und wägen Sie Lesbarkeit und Redundanzfreiheit gegeneinander ab.

Weiterführende Informationen

Siehe [Anforderungen und Ziele](#) in der online-Dokumentation (auf Englisch!).

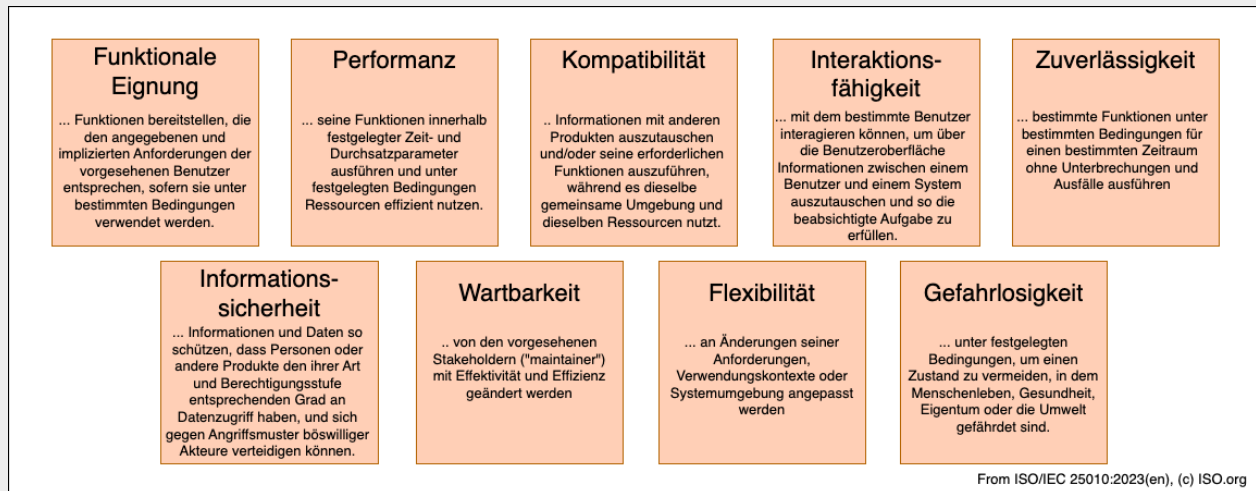
1.2. Qualitätsziele

Inhalt

Die Top-3 bis Top-5 der Qualitätsanforderungen für die Architektur, deren Erfüllung oder Einhaltung den maßgeblichen Stakeholdern besonders wichtig sind. Gemeint sind hier

wirklich Qualitätsziele, die nicht unbedingt mit den Zielen des Projekts übereinstimmen. Beachten Sie den Unterschied.

Hier ein Überblick möglicher Themen (basierend auf dem ISO 25010 Standard):



Motivation

Weil Qualitätsziele grundlegende Architekturentscheidungen oft maßgeblich beeinflussen, sollten Sie die für Ihre Stakeholder relevanten Qualitätsziele kennen, möglichst konkret und operationalisierbar.

Form

Tabellarische Darstellung der Qualitätsziele mit möglichst konkreten Szenarien, geordnet nach Prioritäten.

1.3. Stakeholder

Inhalt

Expliziter Überblick über die Stakeholder des Systems – über alle Personen, Rollen oder Organisationen –, die

- die Architektur kennen sollten oder
- von der Architektur überzeugt werden müssen,
- mit der Architektur oder dem Code arbeiten (z.B. Schnittstellen nutzen),
- die Dokumentation der Architektur für ihre eigene Arbeit benötigen,
- Entscheidungen über das System und dessen Entwicklung treffen.

Motivation

Sie sollten die Projektbeteiligten und -betroffenen kennen, sonst erleben Sie später im Entwicklungsprozess Überraschungen. Diese Stakeholder bestimmen unter anderem Umfang und Detaillierungsgrad der von Ihnen zu leistenden Arbeit und Ergebnisse.

Form

Tabelle mit Rollen- oder Personennamen, sowie deren Erwartungshaltung bezüglich der Architektur und deren Dokumentation.

Rolle	Kontakt	Erwartungshaltung
<Rolle-1>	<Kontakt-1>	<Erwartung-1>
<Rolle-2>	<Kontakt-2>	<Erwartung-2>

Chapter 2. Randbedingungen

Inhalt

Randbedingungen und Vorgaben, die ihre Freiheiten bezüglich Entwurf, Implementierung oder Ihres Entwicklungsprozesses einschränken. Diese Randbedingungen gelten manchmal organisations- oder firmenweit über die Grenzen einzelner Systeme hinweg.

Motivation

Für eine tragfähige Architektur sollten Sie genau wissen, wo Ihre Freiheitsgrade bezüglich der Entwurfsentscheidungen liegen und wo Sie Randbedingungen beachten müssen. Sie können Randbedingungen vielleicht noch verhandeln, zunächst sind sie aber da.

Form

Einfache Tabellen der Randbedingungen mit Erläuterungen. Bei Bedarf unterscheiden Sie technische, organisatorische und politische Randbedingungen oder übergreifende Konventionen (beispielsweise Programmier- oder Versionierungsrichtlinien, Dokumentations- oder Namenskonvention).

Weiterführende Informationen

Siehe [Randbedingungen](#) in der online-Dokumentation (auf Englisch!).

Chapter 3. Kontextabgrenzung

Inhalt

Die Kontextabgrenzung grenzt das System gegen alle Kommunikationspartner (Nachbarsysteme und Benutzerrollen) ab. Sie legt damit die externen Schnittstellen fest und zeigt damit auch die Verantwortlichkeit (scope) Ihres Systems: Welche Verantwortung trägt das System und welche Verantwortung übernehmen die Nachbarsysteme?

Differenzieren Sie fachlichen (Ein- und Ausgaben) und technischen Kontext (Kanäle, Protokolle, Hardware), falls nötig.

Motivation

Die fachlichen und technischen Schnittstellen zur Kommunikation gehören zu den kritischsten Aspekten eines Systems. Stellen Sie sicher, dass Sie diese komplett verstanden haben.

Form

Verschiedene Optionen:

- Diverse Kontextdiagramme
- Listen von Kommunikationsbeziehungen mit deren Schnittstellen

Weiterführende Informationen

Siehe [Kontextabgrenzung](#) in der online-Dokumentation (auf Englisch!).

3.1. Fachlicher Kontext

Inhalt

Festlegung **aller** Kommunikationsbeziehungen (Nutzer, IT-Systeme, ...) mit Erklärung der fachlichen Ein- und Ausgabedaten oder Schnittstellen. Zusätzlich (bei Bedarf) fachliche Datenformate oder Protokolle der Kommunikation mit den Nachbarsystemen.

Motivation

Alle Beteiligten müssen verstehen, welche fachlichen Informationen mit der Umwelt ausgetauscht werden.

Form

Alle Diagrammarten, die das System als Blackbox darstellen und die fachlichen Schnittstellen zu den Nachbarsystemen beschreiben.

Alternativ oder ergänzend können Sie eine Tabelle verwenden. Der Titel gibt den Namen Ihres Systems wieder; die drei Spalten sind: Kommunikationsbeziehung, Eingabe, Ausgabe.

<Diagramm und/oder Tabelle>

<optional: Erläuterung der externen fachlichen Schnittstellen>

3.2. Technischer Kontext

Inhalt

Technische Schnittstellen (Kanäle, Übertragungsmedien) zwischen dem System und seiner Umwelt. Zusätzlich eine Erklärung (*mapping*), welche fachlichen Ein- und Ausgaben über welche technischen Kanäle fließen.

Motivation

Viele Stakeholder treffen Architekturentscheidungen auf Basis der technischen Schnittstellen des Systems zu seinem Kontext.

Insbesondere bei der Entwicklung von Infrastruktur oder Hardware sind diese technischen Schnittstellen durchaus entscheidend.

Form

Beispielsweise UML Deployment-Diagramme mit den Kanälen zu Nachbarsystemen, begleitet von einer Tabelle, die Kanäle auf Ein-/Ausgaben abbildet.

<Diagramm oder Tabelle>

<optional: Erläuterung der externen technischen Schnittstellen>

<Mapping fachliche auf technische Schnittstellen>

Chapter 4. Lösungsstrategie

Inhalt

Kurzer Überblick über die grundlegenden Entscheidungen und Lösungsansätze, die Entwurf und Implementierung des Systems prägen. Hierzu gehören:

- Technologieentscheidungen
- Entscheidungen über die Top-Level-Zerlegung des Systems, beispielsweise die Verwendung gesamthaft prägender Entwurfs- oder Architekturmuster,
- Entscheidungen zur Erreichung der wichtigsten Qualitätsanforderungen sowie
- relevante organisatorische Entscheidungen, beispielsweise für bestimmte Entwicklungsprozesse oder Delegation bestimmter Aufgaben an andere Stakeholder.

Motivation

Diese wichtigen Entscheidungen bilden wesentliche „Eckpfeiler“ der Architektur. Von ihnen hängen viele weitere Entscheidungen oder Implementierungsregeln ab.

Form

Fassen Sie die zentralen Entwurfsentscheidungen **kurz** zusammen. Motivieren Sie, ausgehend von Aufgabenstellung, Qualitätszielen und Randbedingungen, was Sie entschieden haben und warum Sie so entschieden haben. Vermeiden Sie redundante Beschreibungen und verweisen Sie eher auf weitere Ausführungen in Folgeabschnitten.

Weiterführende Informationen

Siehe [Lösungsstrategie](#) in der online-Dokumentation (auf Englisch!).

Chapter 5. Bausteinsicht

Inhalt

Die Bausteinsicht zeigt die statische Zerlegung des Systems in Bausteine (Module, Komponenten, Subsysteme, Klassen, Schnittstellen, Pakete, Bibliotheken, Frameworks, Schichten, Partitionen, Tiers, Funktionen, Makros, Operationen, Datenstrukturen, ...) sowie deren Abhängigkeiten (Beziehungen, Assoziationen, ...)

Diese Sicht sollte in jeder Architekturdokumentation vorhanden sein. In der Analogie zum Hausbau bildet die Bausteinsicht den *Grundrissplan*.

Motivation

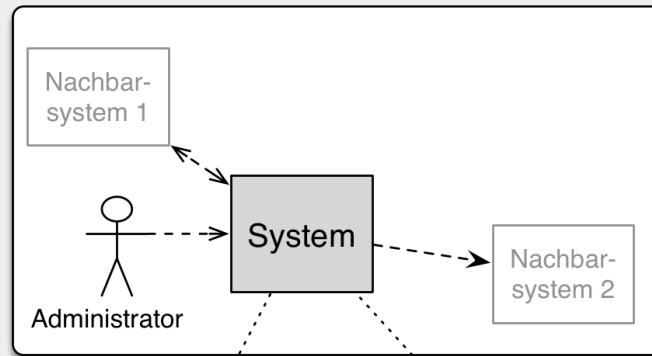
Behalten Sie den Überblick über den Quellcode, indem Sie die statische Struktur des Systems durch Abstraktion verständlich machen.

Damit ermöglichen Sie Kommunikation auf abstrakterer Ebene, ohne zu viele Implementierungsdetails offenlegen zu müssen.

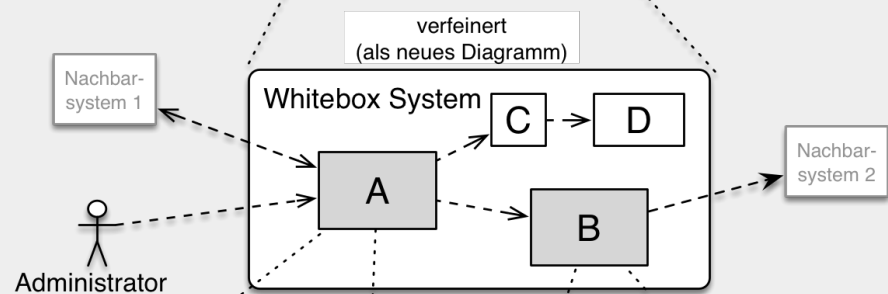
Form

Die Bausteinsicht ist eine hierarchische Sammlung von Blackboxen und Whiteboxen (siehe Abbildung unten) und deren Beschreibungen.

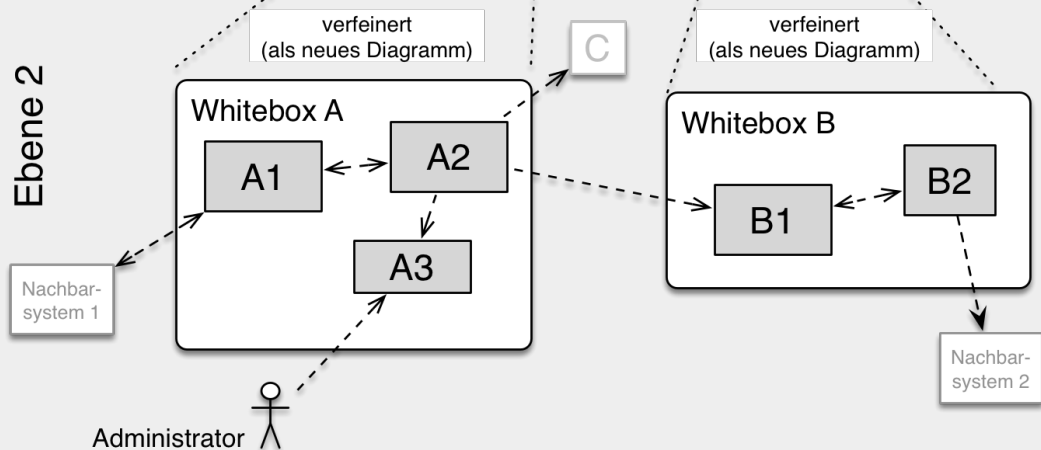
Kontextabgrenzung



Ebene 1



Ebene 2



Ebene 1 ist die Whitebox-Beschreibung des Gesamtsystems, zusammen mit Blackbox-Beschreibungen der darin enthaltenen Bausteine.

Ebene 2 zoomt in einige Bausteine der Ebene 1 hinein. Sie enthält somit die Whitebox-Beschreibungen ausgewählter Bausteine der Ebene 1, jeweils zusammen mit Blackbox-Beschreibungen darin enthaltener Bausteine.

Ebene 3 zoomt in einige Bausteine der Ebene 2 hinein, usw.

Weiterführende Informationen

Siehe [Bausteinsicht](#) in der online-Dokumentation (auf Englisch!).

5.1. Whitebox Gesamtsystem

An dieser Stelle beschreiben Sie die Zerlegung des Gesamtsystems anhand des nachfolgenden Whitebox-Templates. Dieses enthält:

- Ein Übersichtsdiagramm
- die Begründung dieser Zerlegung
- Blackbox-Beschreibungen der hier enthaltenen Bausteine. Dafür haben Sie verschiedene Optionen:
 - in *einer* Tabelle, gibt einen kurzen und pragmatischen Überblick über die enthaltenen Bausteine sowie deren Schnittstellen.
 - als Liste von Blackbox-Beschreibungen der Bausteine, gemäß dem Blackbox-Template (siehe unten). Diese Liste können Sie, je nach Werkzeug, etwa in Form von Unterkapiteln (Text), Unter-Seiten (Wiki) oder geschachtelten Elementen (Modellierungswerkzeug) darstellen.
- (optional:) wichtige Schnittstellen, die nicht bereits im Blackbox-Template eines der Bausteine erläutert werden, aber für das Verständnis der Whitebox von zentraler Bedeutung sind. Aufgrund der vielfältigen Möglichkeiten oder Ausprägungen von Schnittstellen geben wir hierzu kein weiteres Template vor. Im schlimmsten Fall müssen Sie Syntax, Semantik, Protokolle, Fehlerverhalten, Restriktionen, Versionen, Qualitätseigenschaften, notwendige Kompatibilitäten und vieles mehr spezifizieren oder beschreiben. Im besten Fall kommen Sie mit Beispielen oder einfachen Signaturen zurecht.

<Übersichtsdiagramm>

Begründung

<Erläuternder Text>

Enthaltene Bausteine

<Beschreibung der enthaltenen Bausteine (Blackboxen)>

Wichtige Schnittstellen

<Beschreibung wichtiger Schnittstellen>

Hier folgen jetzt Erläuterungen zu Blackboxen der Ebene 1.

Falls Sie die tabellarische Beschreibung wählen, so werden Blackboxen darin nur mit Name und Verantwortung nach folgendem Muster beschrieben:

Name	Verantwortung
<Blackbox 1>	<Text>
<Blackbox 2>	<Text>

Falls Sie die ausführliche Liste von Blackbox-Beschreibungen wählen, beschreiben Sie jede wichtige Blackbox in einem eigenen Blackbox-Template. Dessen Überschrift ist jeweils der Namen dieser Blackbox.

5.1.1. <Name Blackbox 1>

Beschreiben Sie die <Blackbox 1> anhand des folgenden Blackbox-Templates:

- Zweck/Verantwortung
- Schnittstelle(n), sofern diese nicht als eigenständige Beschreibungen herausgezogen sind. Hierzu gehören eventuell auch Qualitäts- und Leistungsmerkmale dieser Schnittstelle.
- (Optional) Qualitäts-/Leistungsmerkmale der Blackbox, beispielsweise Verfügbarkeit, Laufzeitverhalten o. Ä.
- (Optional) Ablageort/Datei(en)
- (Optional) Erfüllte Anforderungen, falls Sie Traceability zu Anforderungen benötigen.
- (Optional) Offene Punkte/Probleme/Risiken

<Zweck/Verantwortung>

<Schnittstelle(n)>

<(Optional) Qualitäts-/Leistungsmerkmale>

<(Optional) Ablageort/Datei(en)>

<(Optional) Erfüllte Anforderungen>

<(optional) Offene Punkte/Probleme/Risiken>

5.1.2. <Name Blackbox 2>

<Blackbox-Template>

5.1.3. <Name Blackbox n>

<Blackbox-Template>

5.1.4. <Name Schnittstelle 1>

...

5.1.5. <Name Schnittstelle m>

5.2. Ebene 2

Beschreiben Sie den inneren Aufbau (einiger) Bausteine aus Ebene 1 als Whitebox.

Welche Bausteine Ihres Systems Sie hier beschreiben, müssen Sie selbst entscheiden. Bitte stellen Sie dabei Relevanz vor Vollständigkeit. Skizzieren Sie wichtige, überraschende, riskante, komplexe oder besonders volatile Bausteine. Normale, einfache oder standardisierte Teile sollten Sie weglassen.

5.2.1. Whitebox <Baustein 1>

...zeigt das Innenleben von *Baustein 1*.

<Whitebox-Template>

5.2.2. Whitebox <Baustein 2>

<Whitebox-Template>

...

5.2.3. Whitebox <Baustein m>

<Whitebox-Template>

5.3. Ebene 3

Beschreiben Sie den inneren Aufbau (einiger) Bausteine aus Ebene 2 als Whitebox.

Bei tieferen Gliederungen der Architektur kopieren Sie diesen Teil von arc42 für die weiteren Ebenen.

5.3.1. Whitebox <_Baustein x.1_>

...zeigt das Innenleben von *Baustein x.1*.

<Whitebox-Template>

5.3.2. Whitebox <_Baustein x.2_>

<Whitebox-Template>

5.3.3. Whitebox <_Baustein y.1_>

<Whitebox-Template>

Chapter 6. Laufzeitsicht

Inhalt

Diese Sicht erklärt konkrete Abläufe und Beziehungen zwischen Bausteinen in Form von Szenarien aus den folgenden Bereichen:

- Wichtige Abläufe oder *Features*: Wie führen die Bausteine der Architektur die wichtigsten Abläufe durch?
- Interaktionen an kritischen externen Schnittstellen: Wie arbeiten Bausteine mit Nutzern und Nachbarsystemen zusammen?
- Betrieb und Administration: Inbetriebnahme, Start, Stop.
- Fehler- und Ausnahmeszenarien

Anmerkung: Das Kriterium für die Auswahl der möglichen Szenarien (d.h. Abläufe) des Systems ist deren Architekturelevanz. Es geht nicht darum, möglichst viele Abläufe darzustellen, sondern eine angemessene Auswahl zu dokumentieren.

Motivation

Sie sollten verstehen, wie (Instanzen von) Bausteine(n) Ihres Systems ihre jeweiligen Aufgaben erfüllen und zur Laufzeit miteinander kommunizieren.

Nutzen Sie diese Szenarien in der Dokumentation hauptsächlich für eine verständlichere Kommunikation mit denjenigen Stakeholdern, die die statischen Modelle (z.B. Bausteinsicht, Verteilungssicht) weniger verständlich finden.

Form

Für die Beschreibung von Szenarien gibt es zahlreiche Ausdrucksmöglichkeiten. Nutzen Sie beispielsweise:

- Nummerierte Schrittfolgen oder Aufzählungen in Umgangssprache
- Aktivitäts- oder Flussdiagramme
- Sequenzdiagramme
- BPMN (Geschäftsprozessmodell und -notation) oder EPKs (Ereignis-Prozessketten)
- Zustandsautomaten
- ...

Weiterführende Informationen

Siehe [Laufzeitsicht](#) in der online-Dokumentation (auf Englisch!).

6.1. <Bezeichnung Laufzeitszenario 1>

- <hier Laufzeitdiagramm oder Ablaufbeschreibung einfügen>
- <hier Besonderheiten bei dem Zusammenspiel der Bausteine in diesem Szenario erläutern>

6.2. <Bezeichnung Laufzeitszenario 2>

...

6.3. <Bezeichnung Laufzeitszenario n>

...

Chapter 7. Verteilungssicht

Inhalt

Die Verteilungssicht beschreibt:

1. die technische Infrastruktur, auf der Ihr System ausgeführt wird, mit Infrastrukturelementen wie Standorten, Umgebungen, Rechnern, Prozessoren, Kanälen und Netztopologien sowie sonstigen Bestandteilen, und
2. die Abbildung von (Software-)Bausteinen auf diese Infrastruktur.

Häufig laufen Systeme in unterschiedlichen Umgebungen, beispielsweise Entwicklung-/Test- oder Produktionsumgebungen. In solchen Fällen sollten Sie alle relevanten Umgebungen aufzeigen.

Nutzen Sie die Verteilungssicht insbesondere dann, wenn Ihre Software auf mehr als einem Rechner, Prozessor, Server oder Container abläuft oder Sie Ihre Hardware sogar selbst konstruieren.

Aus Softwaresicht genügt es, auf die Aspekte zu achten, die für die Softwareverteilung relevant sind. Insbesondere bei der Hardwareentwicklung kann es notwendig sein, die Infrastruktur mit beliebigen Details zu beschreiben.

Motivation

Software läuft nicht ohne Infrastruktur. Diese zugrundeliegende Infrastruktur beeinflusst Ihr System und/oder querschnittliche Lösungskonzepte, daher müssen Sie diese Infrastruktur kennen.

Form

Das oberste Verteilungsdiagramm könnte bereits in Ihrem technischen Kontext enthalten sein, mit Ihrer Infrastruktur als EINE Blackbox. Jetzt zoomen Sie in diese Infrastruktur mit weiteren Verteilungsdiagrammen hinein:

- Die UML stellt mit Verteilungsdiagrammen (Deployment diagrams) eine Diagrammart zur Verfügung, um diese Sicht auszudrücken. Nutzen Sie diese, evtl. auch geschachtelt, wenn Ihre Verteilungsstruktur es verlangt.
- Falls Ihre Infrastruktur-Stakeholder andere Diagrammartarten bevorzugen, die beispielsweise Prozessoren und Kanäle zeigen, sind diese hier ebenfalls einsetzbar.

Weiterführende Informationen

Siehe [Verteilungssicht](#) in der online-Dokumentation (auf Englisch!).

7.1. Infrastruktur Ebene 1

An dieser Stelle beschreiben Sie (als Kombination von Diagrammen mit Tabellen oder Texten):

- die Verteilung des Gesamtsystems auf mehrere Standorte, Umgebungen, Rechner, Prozessoren o. Ä., sowie die physischen Verbindungskanäle zwischen diesen,
- wichtige Begründungen für diese Verteilungsstruktur,
- Qualitäts- und/oder Leistungsmerkmale dieser Infrastruktur,
- Zuordnung von Softwareartefakten zu Bestandteilen der Infrastruktur

Für mehrere Umgebungen oder alternative Deployments kopieren Sie diesen Teil von arc42 für alle wichtigen Umgebungen/Varianten.

<Übersichtsdiagramm>

Begründung

<Erläuternder Text>

Qualitäts- und/oder Leistungsmerkmale

<Erläuternder Text>

Zuordnung von Bausteinen zu Infrastruktur

<Beschreibung der Zuordnung>

7.2. Infrastruktur Ebene 2

An dieser Stelle können Sie den inneren Aufbau (einiger) Infrastrukturelemente aus Ebene 1 beschreiben.

Für jedes Infrastrukturelement kopieren Sie die Struktur aus Ebene 1.

7.2.1. <Infrastrukturelement 1>

<Diagramm + Erläuterungen>

7.2.2. <Infrastrukturelement 2>

<Diagramm + Erläuterungen>

...

7.2.3. <Infrastrukturelement n>

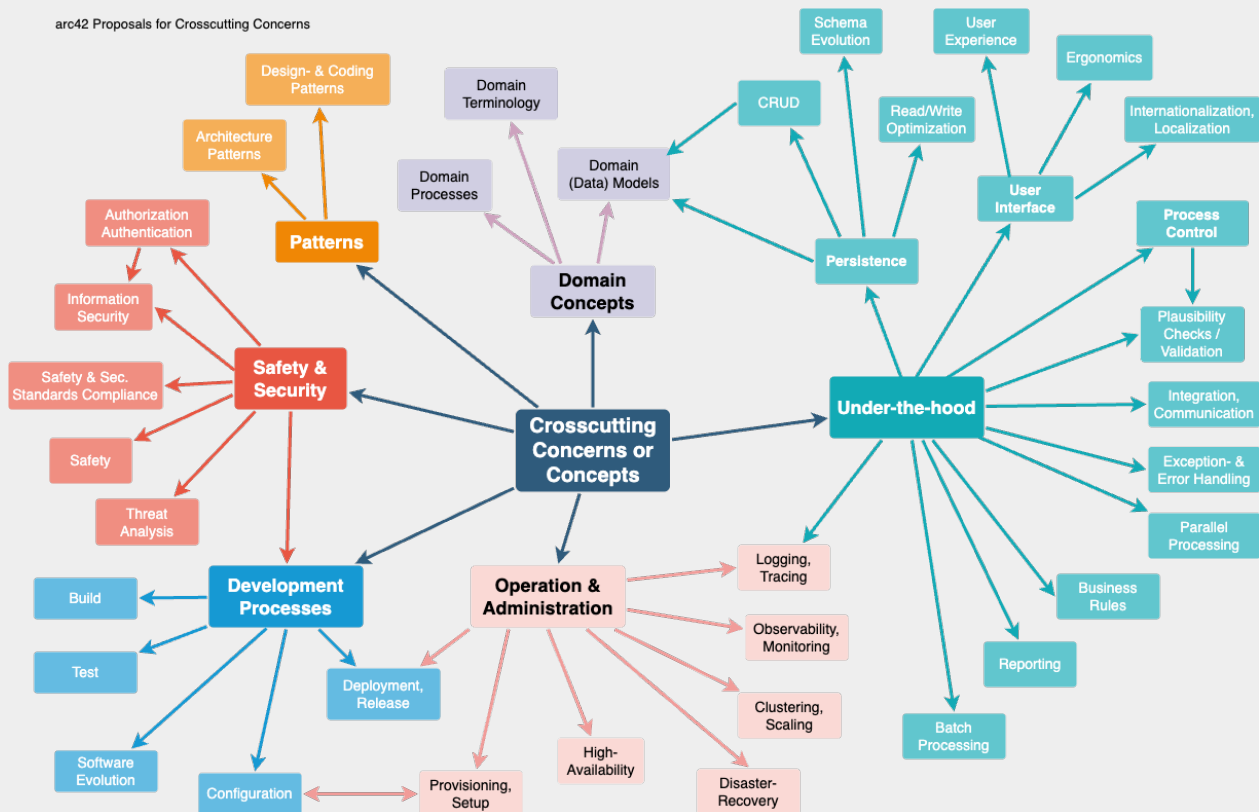
<Diagramm + Erläuterungen>

Chapter 8. Querschnittliche Konzepte

Inhalt

Dieser Abschnitt beschreibt übergreifende, prinzipielle Regelungen und Lösungsansätze, die an mehreren Stellen (=querschnittlich) relevant sind.

Solche Konzepte betreffen oft mehrere Bausteine. Dazu können vielerlei Themen gehören, wie beispielsweise die Themen aus dem nachfolgenden Diagramm:



Motivation

Konzepte bilden die Grundlage für *konzeptionelle Integrität* (Konsistenz, Homogenität) der Architektur und damit eine wesentliche Grundlage für die innere Qualität Ihrer Systeme.

Dieser Abschnitt im Template ist der richtige Ort für die konsistente Behandlung solcher Themen.

Viele solche Konzepte beeinflussen oder beziehen sich auf mehrerer Ihrer Bausteine.

Form

Kann vielfältig sein:

- Konzeptpapiere mit beliebiger Gliederung,
- beispielhafte Implementierung speziell für technische Konzepte,
- übergreifende Modelle/Szenarien mit Notationen, die Sie auch in den Architektursichten nutzen,

Struktur

Wählen Sie **nur** die wichtigsten Themen für Ihr System und erklären das jeweilige Konzept dann unter einer Level-2 Überschrift dieser Sektion (z.B. 8.1, 8.2 etc).

Beschränken Sie sich auf die wichtigen, und versuchen **auf keinen Fall** alle oben dargestellten Themen zu bearbeiten.

Weiterführende Informationen

Einige Themen innerhalb von Systemen betreffen oft mehrere Bausteine, Hardwareelemente oder Prozesse. Es könnte einfacher sein, solche *Querschnittsthemen* an einer zentralen Stelle zu kommunizieren oder zu dokumentieren, anstatt sie in der Beschreibung der betreffenden Bausteine, Hardwareelemente oder Entwicklungsprozesse zu wiederholen.

Bestimmte Konzepte können **alle** Elemente eines Systems betreffen, andere sind vielleicht nur für einige wenige relevant.

Siehe [Querschnittliche Konzepte](#) in der online-Dokumentation (auf Englisch).

8.1. <Konzept 1>

<Erklärung>

8.2. <Konzept 2>

<Erklärung>

...

8.3. <Konzept n>

<Erklärung>

Chapter 9. Architekturentscheidungen

Inhalt

Wichtige, teure, große oder riskante Architektur- oder Entwurfsentscheidungen inklusive der jeweiligen Begründungen. Mit "Entscheidungen" meinen wir hier die Auswahl einer von mehreren Alternativen unter vorgegebenen Kriterien.

Wägen Sie ab, inwiefern Sie Entscheidungen hier zentral beschreiben, oder wo eine lokale Beschreibung (z.B. in der Whitebox-Sicht von Bausteinen) sinnvoller ist. Vermeiden Sie Redundanz. Verweisen Sie evtl. auf Abschnitt 4, wo schon grundlegende strategische Entscheidungen beschrieben wurden.

Motivation

Stakeholder des Systems sollten wichtige Entscheidungen verstehen und nachvollziehen können.

Form

Verschiedene Möglichkeiten:

- ADR ([Documenting Architecture Decisions](#)) für jede wichtige Entscheidung
- Liste oder Tabelle, nach Wichtigkeit und Tragweite der Entscheidungen geordnet
- ausführlicher in Form einzelner Unterkapitel je Entscheidung

Weiterführende Informationen

Siehe [Architekturentscheidungen](#) in der arc42 Dokumentation (auf Englisch!). Dort finden Sie Links und Beispiele zum Thema ADR.

Chapter 10. Qualitätsanforderungen

Inhalt

Dieser Abschnitt enthält alle relevanten Qualitätsanforderungen.

Die wichtigsten davon haben Sie bereits in Abschnitt 1.2 (Qualitätsziele) hervorgehoben, daher soll hier nur auf sie verwiesen werden. In diesem Abschnitt 10 sollten Sie auch Qualitätsanforderungen mit geringerer Bedeutung erfassen, deren Nichterfüllung keine großen Risiken birgt (die aber *nice-to-have* sein könnten).

Motivation

Weil Qualitätsanforderungen die Architekturentscheidungen oft maßgeblich beeinflussen, sollten Sie die für Ihre Stakeholder relevanten Qualitätsanforderungen kennen, möglichst konkret und operationalisiert.

Weiterführende Informationen

- Siehe [Qualitätsanforderungen](#) in der online-Dokumentation (auf Englisch!).
- Siehe auch das ausführliche [Q42 Qualitätsmodell](https://quality.arc42.org) auf <https://quality.arc42.org>.

10.1. Übersicht der Qualitätsanforderungen

Inhalt

Eine Übersicht oder Zusammenfassung der Qualitätsanforderungen.

Motivation

Oft stößt man auf Dutzende (oder sogar Hunderte) von detaillierten Qualitätsanforderungen für ein System. In diesem Abschnitt sollten Sie versuchen, sie zusammenzufassen, z. B. durch die Beschreibung von Kategorien oder Themen (wie z.B. von [ISO 25010:2023](#) oder [Q42](#) vorgeschlagen).

Wenn diese Kurzbeschreibungen oder Zusammenfassungen bereits präzise, spezifisch und messbar sind, können Sie Abschnitt 10.2 auslassen.

Form

Verwenden Sie eine einfache Tabelle, in der jede Zeile eine Kategorie oder ein Thema und eine kurze Beschreibung der Qualitätsanforderung enthält. Alternativ können Sie auch eine Mindmap verwenden, um diese Qualitätsanforderungen zu strukturieren. In der Literatur (insb. [Bass+21]) ist die Idee eines *Quality Attribute Utility Tree* (auf Deutsch manchmal kurz als *Qualitätsbaum* bezeichnet) beschrieben worden, der den Oberbegriff „Qualität“ als Wurzel hat und eine baumartige Verfeinerung des Begriffs „Qualität“ verwendet.

10.2. Qualitätsszenarien

Inhalt

Qualitätsszenarien konkretisieren Qualitätsanforderungen und ermöglichen es zu entscheiden, ob sie erfüllt sind (im Sinne von Akzeptanzkriterien). Stellen Sie sicher, dass Ihre Szenarien spezifisch und messbar sind.

Zwei Arten von Szenarien finden wir besonders nützlich:

- **Nutzungsszenarien** (auch bekannt als Anwendungs- oder Anwendungsfallszenarien) beschreiben, wie das System zur Laufzeit auf einen bestimmten Auslöser reagieren soll. Hierunter fallen auch Szenarien zur Beschreibung von Effizienz oder Performance. Beispiel: Das System beantwortet eine Benutzeranfrage innerhalb einer Sekunde.
- **Änderungsszenarien** beschreiben die gewünschte Wirkung einer Änderung oder Erweiterung des Systems oder seiner unmittelbaren Umgebung. Beispiel: Zusätzliche Funktionalität wird implementiert oder Anforderungen an ein Qualitätsmerkmal ändern sich, und der Aufwand oder die Dauer der Änderung wird gemessen.

Form

Typische Informationen für detaillierte Szenarien sind die folgenden:

In Kurzform (bevorzugt im Q42-Modell):

- **Kontext/Hintergrund:** Um welche Art von System oder Komponente handelt es sich, wie sieht die Umgebung oder Situation aus?
- **Quelle/Stimulus:** Wer oder was initiiert oder löst ein Verhalten, eine Reaktion oder eine Aktion aus.
- **Metrik/Akzeptanzkriterien:** Eine Reaktion einschließlich einer *Maßnahme* oder *Metrik*

Die Langform von Szenarien (die von der SEI und [Bass+21] bevorzugt wird) ist detaillierter und enthält die folgenden Informationen:

- **Szenario-ID:** Ein eindeutiger Bezeichner für das Szenario.
- **Szenario-Name:** Ein kurzer, beschreibender Name für das Szenario.
- **Quelle:** Die Entität (Benutzer, System oder Ereignis), die das Szenario auslöst.
- **Stimulus:** Das auslösende Ereignis oder die Bedingung, auf die das System reagieren muss.
- **Umgebung:** Der betriebliche Kontext oder die Bedingungen, unter denen das System den Stimulus erlebt.
- **Artefakt:** Die Bausteine oder anderen Elemente des Systems, die von dem Stimulus betroffen sind.
- **Reaktion:** Das Ergebnis oder Verhalten, das das System als Reaktion auf den Stimulus zeigt.
- **Antwortmaß:** Das Kriterium oder die Metrik, nach der die Antwort des Systems bewertet

wird.

Beispiele

Ausführliche Beispiele für Qualitätsanforderungen finden Sie auf [der Website zum Qualitätsmodell Q42](#).

Weitere Informationen

- Len Bass, Paul Clements, Rick Kazman: „Software Architecture in Practice“, 4. Auflage, Addison-Wesley, 2021.

Chapter 11. Risiken und technische Schulden

Inhalt

Eine nach Prioritäten geordnete Liste der erkannten Architekturrisiken und/oder technischen Schulden.

Motivation

Risikomanagement ist Projektmanagement für Erwachsene.

— Tim Lister, Atlantic Systems Guild

Unter diesem Motto sollten Sie Architekturrisiken und/oder technische Schulden gezielt ermitteln, bewerten und Ihren Management-Stakeholdern (z.B. Projektleitung, Product-Owner) transparent machen.

Form

Liste oder Tabelle von Risiken und/oder technischen Schulden, eventuell mit vorgeschlagenen Maßnahmen zur Risikovermeidung, Risikominimierung oder dem Abbau der technischen Schulden.

Weiterführende Informationen

Siehe [Risiken und technische Schulden](#) in der online-Dokumentation (auf Englisch!).

Chapter 12. Glossar

Inhalt

Die wesentlichen fachlichen und technischen Begriffe, die Stakeholder im Zusammenhang mit dem System verwenden.

Nutzen Sie das Glossar ebenfalls als Übersetzungsreferenz, falls Sie in mehrsprachigen Teams arbeiten.

Motivation

Sie sollten relevante Begriffe klar definieren, so dass alle Beteiligten

- diese Begriffe identisch verstehen, und
- vermeiden, mehrere Begriffe für die gleiche Sache zu haben.

Form

Zweispaltige Tabelle mit <Begriff> und <Definition>.

Eventuell weitere Spalten mit Übersetzungen, falls notwendig.

Weiterführende Informationen

Siehe [Glossar](#) in der online-Dokumentation (auf Englisch!).

Begriff	Definition
<Begriff-1>	<Definition-1>
<Begriff-2>	<Definition-2>